

Progetto di Programmazione Web e Mobile

A.A. 2025/2026

FastFood — Relazione Tecnica

Progetto Full — svolgimento individuale

Candidato: Matteo Monti

Matricola: 67745A

Docente: prof. Valerio Bellandi

Indice

1. Introduzione
2. Architettura del sistema
3. Struttura e navigazione del sito
4. Presentazione
5. Modello dei dati (MongoDB)
6. Autenticazione e gestione della sessione
7. Realizzazione delle funzionalità richieste
8. API REST e documentazione Swagger
9. Scelte implementative motivate

1. Introduzione

Questo documento descrive la struttura, la presentazione e le scelte implementative dell'applicazione web FastFood, sviluppata come progetto d'esame del corso di Programmazione Web e Mobile (A.A. 2025/2026), nella versione Full svolta individualmente.

FastFood è un sistema di ordinazione online per una catena di ristoranti fast food. Consente a due tipologie di utenti — Cliente e Ristoratore — di registrarsi e gestire rispettivamente l'attività di ordinazione e la gestione del proprio ristorante e del relativo menu.

La relazione è organizzata come segue: la sezione 2 descrive l'architettura generale del sistema; la sezione 3 la struttura e la navigazione delle pagine; la sezione 4 le scelte di presentazione grafica; la sezione 5 il modello dei dati; la sezione 6 il meccanismo di autenticazione; la sezione 7 il dettaglio di realizzazione delle funzionalità richieste dal testo del progetto; la sezione 8 la documentazione delle API REST tramite Swagger; la sezione 9 le scelte implementative motivate; la sezione 10 i limiti noti e gli sviluppi futuri.

2. Architettura del sistema

L'applicazione segue un'architettura client-server con separazione netta tra frontend e backend, comunicanti tramite chiamate REST in formato JSON.

Il frontend è realizzato in HTML5, CSS3 e JavaScript con l'ausilio del framework CSS Bootstrap 5 per la struttura a griglia, i componenti e la responsività. Il foglio di stile personalizzato mystyle.css contiene solo un numero ridotto di regole aggiuntive (centratura verticale del contenuto nelle pagine di login/registrazione, dimensione delle miniature immagine).

Ogni pagina è un file .html indipendente; la logica di interazione (chiamate fetch verso le API, gestione del DOM, validazioni lato client) è inclusa in blocchi `<script>` all'interno della pagina stessa, piuttosto che tramite moduli JS separati o un framework a componenti.

Il backend, invece, è realizzato in Node.js con il framework Express (v5), ed espone esclusivamente API REST che restituiscono/ricevono JSON.

Per l'accesso al database è stato impiegato il driver ufficiale mongodb per Node.js: ogni operazione apre una connessione tramite `MongoClient.connect()`, esegue l'operazione sulla collezione interessata e chiude la connessione.

Il modulo cors è abilitato globalmente per consentire le richieste cross-origin dal frontend (che gira su un'origine diversa da quella del server delle API).

3. Struttura e navigazione del sito

Il sito è organizzato in un'area comune (login e registrazione) e due aree riservate, una per il Cliente e una per il Ristoratore, accessibili solo previa autenticazione. Il controllo di accesso è effettuato lato client, all'apertura di ogni pagina protetta, verificando la presenza e la coerenza dei dati salvati nella `localStorage` — il meccanismo è descritto nel dettaglio nella sezione 6.

3.1 Pagine comuni

Pagina	File	Descrizione
Login	index.html	Form di autenticazione (username, password); reindirizza alla home del Cliente o del Ristoratore in base al tipo di account.
Registrazione	signup.html	Form di registrazione con scelta del tipo di utenza (Cliente/Ristoratore); i campi relativi al metodo di pagamento sono mostrati solo per i clienti.
Account	pages/account.html	Visualizzazione/modifica dei dati del profilo e cancellazione dell'account, comune a entrambe le tipologie di utente.
Dettaglio piatto	pages/dettaglioPiatto.html	Vista di dettaglio di un piatto (nome, categoria, prezzo, ingredienti, ricetta), richiamata sia dal Cliente sia dal Ristoratore.

3.2 Area Cliente

Pagina	File	Descrizione
Home	pages/cliente/cliente.html	Elenco degli ordini attivi (stato "ordinato" o "in preparazione") dell'utente.
Ricerca ristoranti	pages/cliente/ristoranti.html	Elenco di tutti i ristoranti con ricerca per nome/indirizzo.
Ricerca piatti	pages/cliente/piatti.html	Elenco dei piatti di tutti i ristoranti con ricerca per nome/categoria e prezzo massimo; permette di aggiungere piatti al carrello.
Menu ristorante	pages/cliente/menuRistorante.html	Menu del singolo ristorante selezionato, con ricerca per nome/categoria e prezzo massimo; permette di aggiungere piatti al carrello.
Carrello	pages/cliente/carrello.html	Riepilogo degli articoli nel carrello, raggruppati per ristorante, con possibilità di svuotare, cancellare il singolo ordine o confermare l'acquisto.
Storico ordini	pages/cliente/storico.html	Elenco degli ordini con stato "consegnato".

[IMMAGINI C e D]

3.3 Area Ristoratore

Pagina	File	Descrizione
Home	pages/ristoratore/ristoratore.html	Coda degli ordini attivi del proprio ristorante, con azioni per avanzarne lo stato.
Crea ristorante	pages/ristoratore/creaRistorante.html	Form di primo inserimento dei dati del ristorante (mostrato solo se il ristorante non ne possiede ancora uno).
Modifica ristorante	pages/ristoratore/modificaRistorante.html	Modifica o cancellazione dei dati del ristorante già creato.
Menu	pages/ristoratore/menu.html	Elenco dei piatti del proprio menu, con ricerca per nome/categoria, e accesso a dettaglio/modifica.
Aggiungi piatto (da listino)	pages/ristoratore/aggiungiPiatto.html	Selezione di un piatto dal catalogo comune (meal.json) da aggiungere al menu.
Aggiungi piatto (personalizzato)	pages/ristoratore/aggiungiCustom.html	Form libero di inserimento piatto, riutilizzato per completare/correggere i dati di un piatto scelto dal listino comune prima del salvataggio.
Modifica piatto	pages/ristoratore/modificaPiatto.html	Modifica o cancellazione di un piatto del proprio menu.

[IMMAGINE I]

3.4 Logica di navigazione e controlli di accesso

Ogni pagina protetta esegue, al momento del caricamento (evento DOMContentLoaded), un controllo che verifica la presenza in localStorage delle chiavi auth (identificativo dell'utente autenticato) e accType ("cliente" o "ristoratore"); se l'utente non è autenticato o il tipo di account non è coerente con l'area visitata, viene reindirizzato alla pagina di login. Analogamente, le pagine dell'area Ristoratore relative alla gestione del menu verificano la presenza di un ristorante associato all'utente e, se assente, reindirizzano alla pagina di creazione.

4. Presentazione

La presentazione grafica è interamente basata su Bootstrap 5: griglia responsive (row/col), componenti card per la visualizzazione di elenchi di elementi, tabelle per i menu dei ristoranti, navbar responsive con funzionalità specifiche per tipo di account.

Per evitare duplicazione di markup, tutte le pagine che presentano elenchi dinamici (es: ordini) utilizzano lo stesso pattern implementativo: un elemento modello nascosto (classe d-none) definito una sola volta nell'HTML viene clonato via JavaScript (cloneNode) per ogni elemento restituito dalla relativa API, popolato con i dati e reinserito nel DOM.

Il foglio di stile mystyle.css interviene solo dove Bootstrap non copre l'esigenza specifica: centratura verticale del form di login/registrazione (classe centerBody) e dimensionamento delle miniature nella tabella del menu (classe thumb).

5. Modello dei dati (MongoDB)

Il database Fastfood è organizzato in quattro collezioni principali, coerenti con le entità individuate in fase di analisi dei requisiti.

5.1 Collezione user

Campo	Note
_id	generato da MongoDB
nome	
cognome	
username	usato per il login
email	
password	(vedi sezione 9)
tipo	"cliente" oppure "ristoratore"
ncarta	presenti solo per gli utenti di tipo cliente (dati del metodo di pagamento)
datacarta	
cvvcarta	

5.2 Collezione ristorante

Campo	Note
_id	generato da MongoDB
nome	
indirizzo	
città	
telefono	
iva	
id_proprietario	id dell'utente ristorante proprietario

5.3 Collezione piatto

Campo	Note
_id	generato da MongoDB
nome	
categoria	
ricetta	
img	URL dell'immagine illustrativa
prezzo	
ingredienti	ottenuto trasformando in array il testo inserito nel form, separato da virgole
id_ristorante	id del ristorante che offre il piatto

5.4 Collezione ordine

Campo	Note
_id	generato da MongoDB
id_utente	id del cliente che ha effettuato l'ordine
id_ristorante	id del ristorante verso cui è diretto l'ordine
stato	"ordinato" -> "in preparazione" -> "consegnato"
piatti	mappa {idPiatto: quantità}
totale	totale calcolato lato client al momento del checkout
data_creazione	timestamp di creazione dell'ordine
ora_ritiro	orario stimato di ritiro, calcolato in base alla coda
data_consegna	valorizzato dal ristorante quando l'ordine passa a "consegnato"

5.5 Relazioni tra collezioni

Non essendo MongoDB un database relazionale, i riferimenti tra collezioni sono realizzati memorizzando come stringa il campo `_id` del documento referenziato. Questa scelta e l'uso della rappresentazione a stringa anziché del tipo `ObjectId` nativo, sono approfonditi nella sezione 9.

Alla cancellazione di un utente o di un ristorante, il backend effettua un'eliminazione a cascata dei documenti collegati (piatti e ordini del ristorante, ordini del cliente), tramite una funzione di supporto dedicata (`deleteDati`) invocata sulle collezioni interessate.

6. Autenticazione

L'autenticazione è realizzata in modo intenzionalmente semplice: in fase di login, username e password inseriti vengono confrontati direttamente con i valori memorizzati nella collezione `user`; se la combinazione corrisponde a un documento, il backend restituisce l'id dell'utente e il relativo tipo (cliente/ristoratore).

Il frontend memorizza queste informazioni nella `localStorage`, sotto le chiavi `auth` (id utente) e `accType` (tipo utente); queste chiavi vengono lette da ogni pagina protetta per determinare se e come procedere (si veda la sezione 3.4), e vengono azzerate (`localStorage.clear()`) al logout o quando il controllo di accesso fallisce.

7. Realizzazione delle funzionalità richieste

7.1 Gestione del profilo utente

La registrazione (`signup.html`) raccoglie nome, cognome, username, email, password e tipologia di utenza; per i clienti vengono raccolti anche i dati del metodo di pagamento (numero carta, scadenza, CVV). Il form nasconde dinamicamente i campi relativi alla carta quando si seleziona il tipo "Ristoratore".

Il login (`index.html`) autentica l'utente e lo reindirizza alla home corrispondente al proprio tipo di account. La modifica del profilo e la cancellazione dell'account sono centralizzate nella pagina `account.html`, comune a entrambi i tipi di utente, che adatta dinamicamente i campi mostrati (nasconde la sezione dati carta per i ristoratori).

La cancellazione dell'account è realizzata in maniera fisica (hard delete del documento utente), con eliminazione a cascata di ristorante, piatti e ordini collegati, come descritto nella sezione 5.5.

[IMMAGINE [F](#)]

7.2 Gestione del ristorante e del menu

Il ristoratore, dopo la registrazione, viene guidato alla creazione del proprio ristorante (`creaRistorante.html`) se non ne possiede ancora uno; da quel momento può modificarne i dati o cancellarlo (`modificaRistorante.html`).

La composizione del menu prevede due percorsi che convergono nello stesso form finale: (a) selezione di un piatto dal catalogo comune fornito da `meal.json`, i cui dati (nome, categoria, ingredienti, ricetta, immagine) vengono precaricati nel form di inserimento (`aggiungiCustom.html`), lasciando comunque al ristoratore la possibilità di modificarli ed includere il prezzo, prima del salvataggio; (b) inserimento di un piatto interamente personalizzato tramite lo stesso form, senza precompilazione. Questa unificazione dei due percorsi in un solo form evita la duplicazione della logica di validazione e salvataggio.

La modifica e la cancellazione dei singoli piatti sono gestite dalla pagina `modificaPiatto.html`, raggiungibile dall'elenco del menu.

[IMMAGINI [G](#) e [H](#)]

7.3 Ricerca

Le funzionalità di ricerca (ristoranti per nome/luogo; piatti per nome/categoria/prezzo) sono implementate lato client: la pagina scarica l'elenco completo (ristoranti o piatti) tramite le rispettive API REST e, alla sottomissione del form di ricerca, applica un filtro sul DOM già popolato (confronto case-insensitive su nome/categoria e confronto numerico sul prezzo), nascondendo gli elementi che non soddisfano il filtro anziché effettuare una nuova richiesta al server con parametri di ricerca.

7.4 Gestione degli ordini

Il carrello è mantenuto interamente lato client in localStorage (chiave carrello), come oggetto che associa a ciascun ristorante un insieme di coppie piatto-quantità; i pulsanti +/- nelle pagine di ricerca piatti e menu ristorante aggiornano direttamente questa struttura. Il carrello può contenere articoli di più ristoranti contemporaneamente: in fase di checkout viene generato un ordine distinto per ciascun ristorante presente nel carrello.

Il calcolo del tempo di attesa stimato avviene al momento della conferma dell'ordine (funzione `ordina()` in `carrello.html`): il sistema interroga il numero di ordini già in coda (stato "ordinato" o "in preparazione") per quel ristorante e stima un'attesa di 10 minuti per ogni ordine in coda, sommata a un tempo base di 10 minuti.

Il flusso di stato realizzato è: ordinato -> in preparazione -> consegnato. Il ristorante, dalla propria home, avanza lo stato dell'ordine con due azioni dedicate ("In preparazione" e "Consegnato"); quando l'ordine passa a "consegnato" viene registrata anche la data/ora effettiva. Non essendo prevista la consegna a domicilio, lo stato intermedio "in consegna" definito nella specifica non viene mai raggiunto nell'implementazione realizzata.

Il cliente visualizza gli ordini attivi nella propria home (`cliente.html`) e lo storico degli ordini conclusi nella pagina dedicata (`storico.html`), interrogando la collezione ordine filtrata per utente e stato.

[IMMAGINE E]

8. API REST e documentazione Swagger

Tutte le operazioni sopra descritte sono esposte dal backend come endpoint REST, documentati automaticamente tramite la libreria `swagger-autogen`, che analizza le route definite in `main.js` e genera il file `swagger.json`; quest'ultimo viene poi servito tramite `swagger-ui-express`, rendendo la documentazione interattiva disponibile all'indirizzo `http://localhost:3000/swagger` quando il server è in esecuzione.

Metodo	Endpoint	Descrizione
POST	/login	Autentica un utente tramite username e password
POST	/user	Registra un nuovo utente (cliente o ristorante)
GET	/user/{id}	Recupera i dati di un utente
PUT	/user/{id}	Modifica i dati di un utente
DELETE	/user/{id}	Cancella un utente e i dati collegati
POST	/ristorante/{ristoratore}	Crea il ristorante per il ristoratore indicato
GET	/ristorante/{ristoratore}	Recupera il ristorante di un ristoratore
PUT	/ristorante/{ristoratore}	Modifica i dati del ristorante
DELETE	/ristorante/{ristoratore}	Cancella il ristorante e i dati collegati
GET	/ristoranti/	Elenca tutti i ristoranti
GET	/ristoranti/{id}	Recupera un singolo ristorante per id
POST	/piatto/{ristorante}	Aggiunge un piatto al menu del ristorante
GET	/piatti/{ristorante}	Elenca i piatti di un ristorante
GET	/piatto/{id}	Recupera un singolo piatto
PUT	/piatto/{id}	Modifica un piatto
DELETE	/piatto/{id}	Cancella un piatto
POST	/ordini/	Elenca gli ordini secondo un filtro passato nel body
POST	/ordine/	Crea un nuovo ordine
PUT	/ordine/{id}	Aggiorna lo stato di un ordine

Si osserva che l'operazione di ricerca degli ordini (`/ordini/`) utilizza il metodo POST anziché GET: la scelta è motivata nella sezione 9.

[IMMAGINE J]

9. Motivazione scelte implementative

- Riferimenti tra collezioni come stringa dell'ObjectId: per evitare la necessità di conversioni ripetute tra tipo ObjectId e stringa nelle interrogazioni e nel confronto lato client (dove l'id arriva sempre come stringa da JSON), i campi di relazione (es. `id_ristorante`, `id_proprietario`) sono stati modellati come stringa anziché come ObjectId.
- Ricerca lato client: per ristoranti e piatti, la ricerca filtra gli elementi già scaricati e visualizzati nel DOM, invece di introdurre parametri di query lato server, mantenendo la struttura più compatta e semplice.
- Carrello e localStorage: il carrello è mantenuto nella localStorage piuttosto che sul database, riducendo la complessità del backend ed evitando query massive per il continuo aggiornamento dei dati.

- Unificazione del form di inserimento piatto: sia il piatto scelto dal catalogo comune sia il piatto personalizzato confluiscono nello stesso form (aggiungiCustom.html), riducendo la duplicazione di codice e permettendo comunque al ristoratore di correggere/integrare i dati precompilati dal catalogo.
- Uso di POST per la ricerca ordini: la scelta è motivata dalla necessità di trasmettere un filtro strutturato (es. combinazioni di stato tramite operatore \$or di MongoDB) nel corpo della richiesta, evitando la costruzione di query string complesse; è una deviazione consapevole rispetto alla convenzione REST.
- Le password sono memorizzate in chiaro nel database, senza alcun meccanismo di hashing